Effective Agentic Coding for Scientific Computing

The code is the easy part: asking the questions, checking the answers, and training who does both

---

James Kermode · School of Engineering, University of Warwick

27 October 2026 · CECAM Mary Ann Mansigh Lecture (online)

Personal views as a Warwick academic and HetSys CDT director. The HetSys AI policy shown is the CDT's adopted policy; the rest is not the institutional view of the University of Warwick.

The end of human programming?

> *“The end of human programming?”*
> *“At the VASP Software GmbH, about 70% of our team now uses multi-agent LLM pools to develop software.”*
>
> — G. Kresse, CECAM abstract, 27 Oct 2026

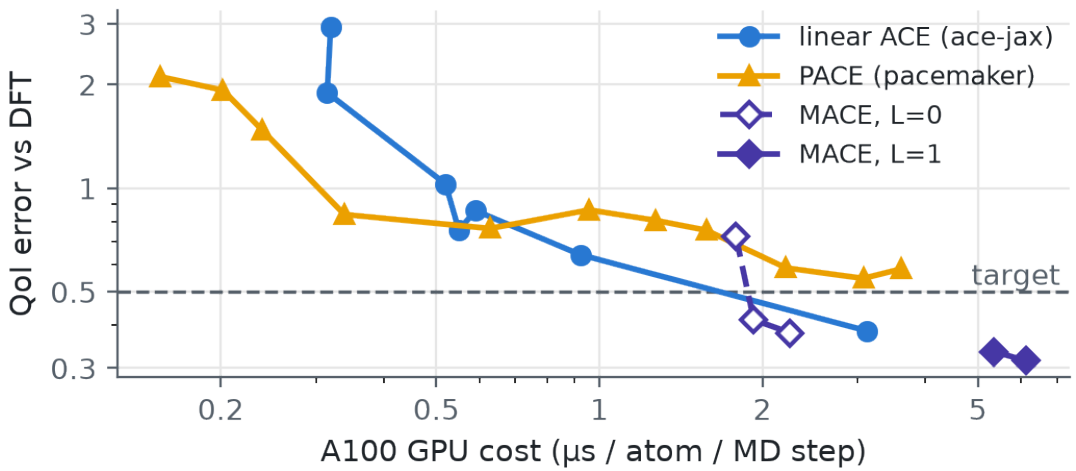
Code now arrives faster than humans can review it, so validation and regression testing are handed to AI too.

If validation relies on AI, the human skill that is left is knowing what to validate against and which question to ask.

Where does the next generation learn that?

ace-jax: the agent wrote the code, every test was green

- JAX port of ACE interatomic potentials, CPU and GPU
- 107 of 109 commits agent co-authored; ≈20k lines of source, ≈28k of tests
- Parity in CI against ACEpotentials.jl and ML-PACE / python-ace, at 10^{-12}–10^{-14}
- Human work: choosing the references, the tolerances, the benchmarks, and certifying the result



`pip install ace-jax` · evidence-chosen fit weights · calibrated force uncertainties · pacemaker `.yace` in/out

Bug #57: basis-builder columns misaligned. Every test was green: the Julia parity CI checked the evaluator on a model Julia had built, never ace-jax's own builder.

Fix #70: built-basis parity against Julia (agrees to 2×10^{-14}; without the fix, 109/110 blocks fail), and a missing reference now fails instead of skipping.

Data: GAP-18 (Bartók et al., Phys. Rev. X 8, 041048, 2018). Parity reference: ACEpotentials.jl (Witt … Ortner, J. Chem. Phys. 159, 164101, 2023). LAMMPS: lammps-jax (Gangan, Witt & Kotak, 2026). Compared: pacemaker (Bochkarev et al., Phys. Rev. Mater. 6, 013804, 2022); MACE (Batatia et al., NeurIPS 2022) run by Symmetrix (W. C. Witt; Kovács et al., J. Am. Chem. Soc. 147, 17598, 2025).

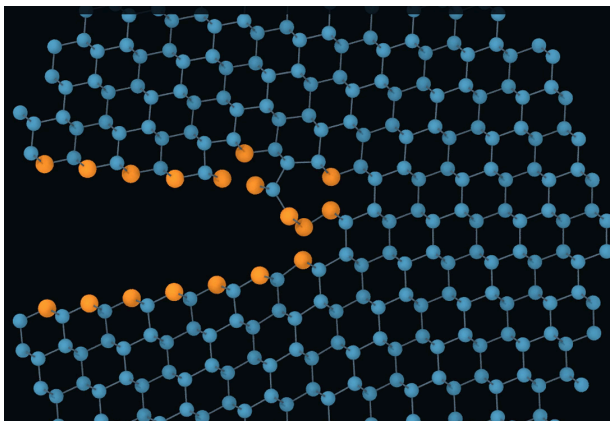
github.com/ACEsuit/ace-jax · acesuit.github.io/ace-jax · benchmarks

Benchmark task: once fully specified, difficulty collapses

Terminal-Bench-Science `si-fracture-fbc`: Si(111) crack tip, Stillinger–Weber, flexible boundary + Cauchy–Born corrector. Implement from scratch; locate the lattice-trapping folds.

Version	Change	Result
v0.1 (Aug)	original task	previous generation: no valid pass
Repaired	corrected oracle (my bug)	GPT-6 Astra: all 9 gates, 21 min, $6.63
Hardened	mixed-mode crossover	Astra: all 7 gates, 65 min
Hardened again	reconstructed-branch fold	Opus 5.5 ✓, Astra ✓, ≈1 h, $7–13
3D follow-up	crack-front kinks, free energy at 600 K (≈40 h expert task)	Opus 5.5 ✓, ≈1.5 h, key physics unprompted

Labs now headline it: Opus 5 29.0% → Opus 5.5 58.7%; GPT-5.6 Sol 22.4% → GPT-6 Astra 64.6% (Opus 5.5 System Card §8.6; OpenAI GPT-6 Astra launch)



Reference solution at the upper fold (Stillinger–Weber), the last trapped state before the crack advances one lattice period. Gold: under-coordinated atoms.

Once a task is fully specified, the difficulty collapses. The human work is choosing the question.

terminal-bench-science.ai · task: si-fracture-fbc

Where did we learn to do this?

Asking the question

The benchmark: specification is now the hard part; implementation is not.

Deciding what counts as correct

ace-jax: consistency checks are not correctness checks. Green CI ≠ right answer.

These skills came from struggle: years of debugging, deriving, and being wrong.

Without struggle-as-pedagogy, they will not be automatic.

Patterns and anti-patterns

Patterns that work

- Hoard things you know how to do
- Spec first, implement second
- Red/green TDD
- Manage context obsessively

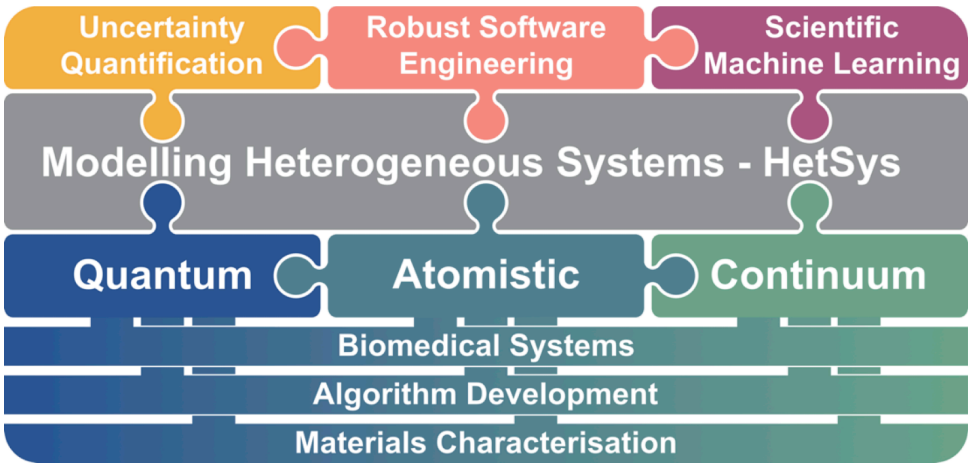
Anti-patterns

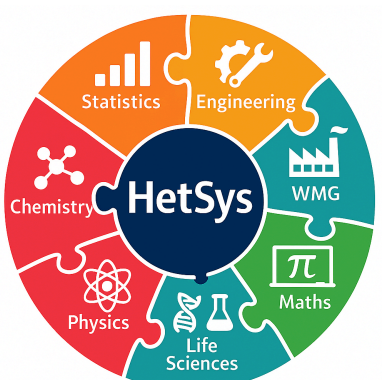
- Unreviewed code on collaborators
- Trusting output you can’t verify
- Stateful notebooks
- Over-engineering by default

Full talk, with the evidence for each:
github.com/jameskermode/agentic-coding-seminar

HetSys AI policy (v1.0, Aug 2026)

- Skill erosion is the central concern, not only rules
- Build foundations by hand in PhD years 1–2: “the friction is educational”
- Keep one regular problem you solve without AI
- Tests use numerical tolerances you choose, not the AI
- Watch for “verification theatre”: code that looks verifiable but isn’t
- Declare substantive use — supervisors too
- Everyday test: would the same help from a colleague be acceptable?





Assessing understanding, not authorship

Not all MSc/PhD code will be human-written. mograder separates what a machine can check from what a human must.

Formative — the code

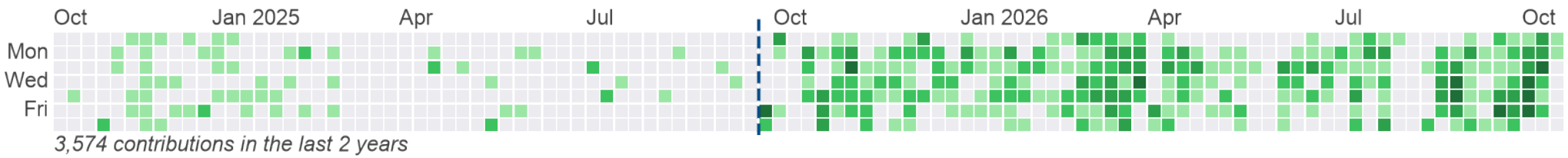
- Automated checks run as each cell is completed
- Partial credit and targeted feedback, fast turnaround
- Low stakes

Summative — the understanding

- **Random mini-vivas**: 10–15 min walking a marker through the submitted code; sample allocated after submission
- Their name is on it: however it was written, they defend it

Delegate pattern-matching — did the code produce the right answer? — to the machine; keep the human question — did the student understand? — for humans.

Discussion



My commits: 2,332 Claude co-authored, across 52 repositories (Oct 2025 – Oct 2026).

Who will train the researchers of the future, if we outsource all coding to agents now?

Full talk, slides + source

github.com/jameskermode/agentic-coding-seminar

Acknowledgements: Claude usage supported by credits from Anthropic's AI for Science program. HetSys is funded by EPSRC (EP/S022848/1, EP/Y035429/1). Thanks to Karsten Jacobsen (DTU Physics) for sharing the AI guidelines that were the starting point for the HetSys policy, and to the HetSys staff and students who co-created it. Thanks to the Terminal-Bench-Science team and reviewers. Benchmarks ran on Sulis (EPSRC EP/T022108/1, HPC Midlands+).

UKRI Engineering and Physical Sciences Research Council